

FINAL YEAR PROJECT — PFE 2026

MobiCloud — Big Data Questions

Interview & Defense Preparation — Distributed Storage on Mobile P2P Networks

 Author: Anis Naoui  June 2026

 Source: MobiCloud Android App + Relay Server + PeerSim Simulation

30

Total Questions

7

Thematic Sections

10

Hard-level Questions

12

Medium-level Questions

8

Easy-level Questions



SECTION 1 — DISTRIBUTED STORAGE & ERASURE CODING

5 questions

1

Why did you choose Reed-Solomon erasure coding over simple block replication for MobiCloud?

Medium

Reed-Solomon

Fault Tolerance

Storage Efficiency

♦ MODEL ANSWER

Simple replication (k copies) introduces a storage overhead of $k\times$. With Reed-Solomon $RS(k=3, n=2)$, a file is split into 3 data blocks and 2 parity blocks, producing only **5 blocks total**, meaning a storage overhead of $5/3 \approx 1.67\times$ — compared to $3\times$ with triple replication for the same fault tolerance (2 node failures). On mobile devices where storage is scarce, this difference is significant.

- Simple replication ($k=3$): $3\times$ overhead, tolerates 2 failures
- $RS(k=3, n=2)$: $1.67\times$ overhead, tolerates 2 failures — **same resilience, 45% less storage**
- Recovery: any $K=3$ blocks out of 5 are sufficient to reconstruct the file
- Constraint from the code: $k + n \leq 255$ (GF(256) arithmetic limit)

```
// From DEFENSE_PREP.md / EncodeErasureFragmentsUseCase.kt
// File → zero-padded → split into K data blocks
// Codec (Reed-Solomon GF(256)) → N parity blocks
// Recovery: any K out of K+N blocks → full reconstruction
```

2

Explain the complete file upload pipeline in MobiCloud from the moment the user taps "Store file" to DHT propagation.

Hard

Pipeline

Erasure Coding

AES-GCM

DHT

Gossip

✦ MODEL ANSWER

The upload pipeline follows 7 sequential stages, each implemented as a dedicated use case:

User taps "Store file"

- ↓ `ExplorerViewModel.store()`
- ↓ `EncodeErasureFragmentsUseCase` → K data + N parity fragments (Reed-Solomon GF(256))
- ↓ `FragmentCipherUseCase` → AES-256-GCM encrypt each fragment
(HKDF-SHA256 derives a per-fragment key)
- ↓ `ConsistentHashRing` → assigns each fragment to a responsible peer node
- ↓ `BlockTransferChannel` (TCP) → sends encrypted fragment to target peer
(falls back to WebSocket relay if NAT)
- ↓ `InsertDhtEntryUseCase` → logs `blockId` → `nodeId` in local DHT
- ↓ `Gossip` (M03-04) → `BloomFilter` delta-sync propagates DHT entry

- The relay server (`server.js`) handles `UPLOAD` (opcode 0x06) with store-and-forward buffering
- Zero-knowledge: encrypted blocs pass through the relay opaque — the relay never sees plaintext
- Download reverses the pipeline: lookup DHT → fetch K fragments → Reed-Solomon decode → AES-GCM decrypt

3

Why does the erasure codec work over GF(256) specifically? What is the significance of this Galois Field?

Hard

GF(256)

Galois Field

Mathematics

JNI / C++

♦ MODEL ANSWER

Reed-Solomon codes require arithmetic in a **finite field (Galois Field)**. GF(256) = GF(2^8) was chosen because:

- $256 = 2^8$ maps **exactly to one byte** — each field element is a single byte, making implementation natural for binary data
- Arithmetic (addition, multiplication, inversion) is performed at the byte level using lookup tables in C++
- Constraint: $k + n \leq 255$ because GF(256) has 256 elements (0–255), and the Vandermonde matrix requires distinct non-zero evaluation points
- The codec is implemented in native C++ and called from Kotlin via JNI (`EraseCodingJni`) using a `DirectByteBuffer` for zero-copy data transfer

Alternative fields (GF(2^{16})) would support larger $k+n$ but double the per-element size and slow down operations.

4

How does MobiCloud handle automatic data repair when a peer goes offline permanently?

Hard

Self-Healing

Repair Protocol

Circuit Breaker

✦ MODEL ANSWER

MobiCloud implements a proactive repair mechanism orchestrated by the Super-Peer:

- **Detection (10s scan):** The Super-Peer scans every 10 seconds for under-replicated blocks (blocks with fewer than $K+N$ live replicas)
- **Graceful departure:** A node sends a `DepartureNoticeMessage` → `DepartureNoticeHandler` immediately identifies orphaned blocks without waiting for the 10s scan
- **Migration planning:** `LocalRepairBuffer` queues re-replication tasks; `ConsistentHashRing` assigns orphaned blocks to the next responsible node
- **Cryptographic signing:** The repair plan is signed with EC P-256 (anti-replay window $\pm 30s$) to prevent malicious migration directives
- **Execution:** `BlockTransferChannel` migrates fragments to new hosts via TCP or relay
- **Circuit Breaker:** If churn $> 30\%$ in a 5-minute sliding window, repair is suspended to avoid a repair storm; resumes when churn drops below 10%

5

What does the relay server's `relayBuffer` store and why is it bounded to 500 entries?

Medium

Store-and-Forward

Memory Management

NAT Traversal

◆ MODEL ANSWER

The `relayBuffer` in `server.js` is a `Map<blockId, [{fromNodeId, destNodeId, data, ttlTimer}]>` that temporarily stores encrypted block fragments when the destination peer is offline at delivery time.

- Cap of **500 entries** (`MAX_RELAY_BUFFER_ENTRIES`) prevents unbounded RAM consumption on the Node.js relay server — the relay is stateless by design
- Each entry has a **TTL of 3 minutes** (180,000 ms) — sufficient to survive Android Doze mode pauses (~90s)
- When the destination peer reconnects, `flushPendingBlocks()` delivers buffered blocks immediately
- Deduplication: if `(blockId, destNodeId)` already exists in the buffer, the old entry is replaced
- The relay stores **only encrypted ciphertext** — zero-knowledge property is preserved

```
// server.js line ~41
const MAX_RELAY_BUFFER_ENTRIES = 500;
const TTL_MS = 180_000; // 3 min – outlasts Android Doze (~90s)
```



SECTION 2 — DHT, GOSSIP PROTOCOL & CONSISTENT HASHING

5 questions

6

Explain the Bloom Filter gossip protocol used in MobiCloud. Why not send the full DHT catalog to each peer?

Hard

Bloom Filter

Delta-Sync

Bandwidth Efficiency

✦ MODEL ANSWER

Sending the full DHT catalog is $O(\text{entries} \times \text{entry_size})$ per gossip round — unsustainable on mobile networks. MobiCloud uses a **1024-bit Bloom Filter with 3 SHA-256-based hash functions** as a compact summary:

- **add(blockId)**: sets 3 bits determined by 3 hash functions; **mightContain(blockId)**: checks those 3 bits
- False positives are possible (rare) — false negatives are **impossible**
- A node sends its 128-byte BloomFilter to peers → peers identify entries *not* in the filter → send only those delta entries
- Bandwidth is $O(\text{delta})$ instead of $O(\text{full catalog})$ — critical on 4G/mobile
- Pre-allocated `MessageDigest` instances avoid 300 `getInstance()` calls per gossip round (optimization noted in the codebase)

```
// MobiCloudGossipDht.java (PeerSim model – simplified, no Bloom filter)
// Real Android code: GossipSyncUseCase.runGossipCycle()
// Every 2 seconds: send BloomFilter(128 bytes) to fanout=2 peers
// Peer responds with delta entries NOT in the filter
```

7

How does the Consistent Hash Ring in MobiCloud map a file block to a responsible peer node?

Medium

Consistent Hashing

Load Balancing

DHT

✦ MODEL ANSWER

The `ConsistentHashRing` maps a `blockId` to a node using a single deterministic computation:

```
// ConsistentHashRing.kt
// blockId (string) → SHA-256 → take first 4 bytes → big-endian int
// responsibleNode = floorMod(hash4bytes, numNodes)

blockId → SHA-256 → first 4 bytes → int → floorMod(int, N) → nodeIndex
```

- The key advantage of consistent hashing: when a node joins or leaves, only **1/N fraction** of blocks need redistribution (vs. full reshuffle with modulo hashing)
- SHA-256 ensures **uniform distribution** across all nodes
- The same ring is used for repair: after a departure, orphaned blocks are reassigned to the *next* responsible node on the ring

CRDT

Tombstone

Conflict Resolution

✦ MODEL ANSWER

A **tombstone** is a special DHT entry that marks a block as *logically deleted* (with a timestamp). Without tombstones:

- A peer deletes a block locally, then goes offline
- It rejoins and gossips its DHT — the deleted entry **re-appears** because the deletion was never propagated
- Tombstones carry the deletion timestamp → gossip Last-Writer-Wins (LWW) resolution keeps the delete if it is more recent than any re-insert
- MobiCloud tombstones expire after **1 hour** to avoid unbounded storage
- Implemented as a CRDT (Conflict-free Replicated Data Type): merge is commutative, associative, idempotent — no central coordination needed

```
// MobiCloudGossipDht.java – mergeDht() implements LWW
private static void mergeDht(Map<Integer, DhtEntry> target, Map<Integer, DhtEntry> source) {
    for (Map.Entry<Integer, DhtEntry> e : source.entrySet()) {
        DhtEntry existing = target.get(e.getKey());
        // Keep entry with HIGHER timestamp (Last-Writer-Wins)
        if (existing == null || e.getValue().timestamp > existing.timestamp) {
            target.put(e.getKey(), e.getValue());
        }
    }
}
```

9

What convergence time did your PeerSim simulation show for the gossip DHT, and what does $O(\log N)$ convergence mean in practice for 1000 nodes?

Medium

PeerSim

Simulation

Scalability

✦ MODEL ANSWER

The PeerSim simulation (`MobiCloudGossipDht.java` + `ConvergenceObserver.java`) demonstrated epidemic convergence:

- With **fanout=2** (each node gossips with 2 random neighbors per cycle), the DHT converges in $O(\log N)$ cycles
- For $N=1000$: $\log_2(1000) \approx 10$ cycles → at 2 seconds per cycle → **~20 seconds** to full convergence
- For $N=10,000$: $\log_2(10000) \approx 13$ cycles → ~26 seconds — the convergence scales gracefully
- Even under 30% churn (nodes randomly going offline), the gossip remained robust (results: `churn30-results.csv`)
- fanout=2 is the optimal trade-off: fanout=5 converges in 2 cycles but **drains battery 2.5× faster** (BatteryProtocol simulation)

10

How does the relay server route a SIGNAL message for gossip between two peers that are both online?

Easy

WebSocket

Gossip Relay

Opcode

✦ MODEL ANSWER

The SIGNAL opcode (0x0E) in `server.js` handles ephemeral P2P forwarding (used for DHT gossip between peers):

```
// server.js - handleSignal()
// Payload: 16 bytes destNodeId (UTF-8, NUL-padded) + arbitrary data
function handleSignal(fromNodeId, payload) {
  const destNodeId = payload.slice(0, 16).toString('utf8').replace(/\0+$/, '').trim();
  const data = payload.slice(16);
  const destSession = sessions.get(destNodeId);
  if (destSession && destSession.ws.readyState === WebSocket.OPEN) {
    // Forward immediately - NO buffering
    safeSend(destSession.ws, buildFrame(MSG.SIGNAL_RECEIVED, out));
    eventCounters.signalsSent++;
  } else {
    eventCounters.droppedSignals++; // signal dropped if dest offline
  }
}
```

→ **No buffering:** signals are fire-and-forget — if the destination is offline, the signal is dropped (gossip is self-healing anyway)

→ Contrasts with UPLOAD (0x06) which *does* buffer in `relayBuffer`



SECTION 3 — SECURITY, CRYPTOGRAPHY & IDENTITY

5 questions

11

How does MobiCloud prevent Sybil attacks? What does "18-bit difficulty" mean in terms of computational cost?

Medium

Hashcash PoW

Anti-Sybil

SHA-256

✦ MODEL ANSWER

MobiCloud uses a **Hashcash Proof-of-Work** (M01 module) to make mass identity creation computationally expensive:

- A node must find a nonce such that SHA-256(`nodeId:timestamp:nonce`) starts with **18 zero bits**
- Expected iterations: $2^{18} = 262,144$ SHA-256 hashes $\approx \sim 1$ **second on ARM mobile**
- A Sybil attacker wanting to create 1000 fake identities must invest ~ 1000 CPU-seconds — economically infeasible at scale
- The PoW token is cached → avoids recomputation on reconnect
- A signature is appended to prove the PoW was done by *this* node's private key (Android Keystore)

```
// GenerateHashcashProofUseCase.kt (M01)
// CPU loop: nodeId:timestamp:nonce → SHA-256 → check 18 leading zero bits → nonce++
// Difficulty = 18 bits → E[iterations] =  $2^{18} \approx 262,144 \approx \sim 1s$  on ARM
```

12

Explain how a node's identity (nodeId) is generated and why it cannot be cloned or forged.

Easy

Android Keystore

EC P-256

TEE / StrongBox

✦ MODEL ANSWER

Node identity generation:

Android Keystore (TEE/StrongBox) → generate EC P-256 key pair
→ export public key (SPKI-DER format)
→ $\text{SHA-256}(\text{publicKey})[0..7] \rightarrow \text{nodeId} = 16\text{-character hex string}$

- The private key is generated inside the **Trusted Execution Environment (TEE)** or StrongBox and *never leaves the hardware*
- It is **device-bound**: cannot be extracted, exported, or cloned — even root access cannot read it
- Identity is **persistent across app reinstalls** (Android Keystore survives reinstallation)
- The relay verifies identity at every connection with EC P-256 signature + 30-second anti-replay window
- nodeId collision probability: $2^{(-64)}$ — negligible for mobile network sizes

13

Describe the encryption pipeline for a single file fragment in MobiCloud. What does "zero-knowledge relay" mean?

Hard

AES-256-GCM

HKDF-SHA256

ECIES

Zero-Knowledge

◆ MODEL ANSWER

Each fragment receives its own derived encryption key:

File → generate random 256-bit master key

Master key + fragment_index → HKDF-SHA256 → per-fragment key (256 bits)

Fragment plaintext + per-fragment key → AES-256-GCM → ciphertext + 128-bit auth tag

Master key → ECIES wrap: ECDH(sender_priv, recipient_pub) + HKDF + AES → wrapped key

Stored alongside metadata: blockId, ownerId, encryptedMasterKey

- **HKDF-SHA256 (RFC 5869)**: derives a unique key per fragment from the master key — compromise of one fragment key does not expose others
- **AES-256-GCM**: authenticated encryption — provides both confidentiality and integrity
- **Zero-knowledge relay**: the relay server (`server.js`) stores and forwards `payload.slice(80)` (the raw ciphertext) without any decryption. Even with full relay access, an attacker cannot read the data
- Comment in `server.js`: *"bloc chiffré AES-256 GCM — JAMAIS transformé (AC#4 Zero-Knowledge)"*

14

How does the relay server authenticate connecting nodes? What prevents replay attacks?

Medium

EC P-256

Anti-Replay

AUTH Protocol

♦ MODEL ANSWER

Authentication uses opcode AUTH (0x01) with a challenge-response mechanism:

```
// verifyAuth() in server.js
```

```
Client sends: { nodeId, pubKeySpkiDer (base64), timestamp (Unix ms), signature (base64) }
```

```
Server verifies:
```

1. nodeId format: `/^[0-9a-fA-F]{16}$/` (16 hex chars)
2. timestamp within 30-second window → prevents replay (`AUTH_WINDOW_MS = 30_000`)
3. pubKey is EC P-256 (prime256v1) – curve validated
4. signature = `ECDSA(SHA-256, "MobiCloud-HA-AUTH:nodeId:timestamp")`

- **Anti-replay:** timestamp must be within ± 30 seconds of server time; a captured token becomes invalid after 30 seconds
- **AUTH timeout:** if a client doesn't authenticate within 10 seconds, the connection is closed
- nodeId must be exactly 16 hex chars (ASCII only, no multi-byte UTF-8 bypass possible)

15

What is ECIES and why is it used in MobiCloud's key wrapping scheme?

Hard

ECIES

ECDH

Key Wrapping

♦ MODEL ANSWER

ECIES (Elliptic Curve Integrated Encryption Scheme) allows encrypting data to a recipient knowing only their public key:

- **Why ECIES?** The master encryption key must be stored so the *file owner* can later decrypt. AES alone can't do asymmetric access control — ECIES enables encrypting the AES master key to the owner's EC P-256 public key
- **Mechanism:** ECDH(ephemeral_priv, recipient_pub) → shared secret → HKDF → wrapping key → AES-encrypt master key
- Only the holder of the EC P-256 private key (in Android Keystore TEE) can unwrap the master key
- Cryptographic table: AES-256-GCM (fragment encryption), HKDF-SHA256 (key derivation), EC P-256 (identity + ECIES), SHA-256 with ECDSA (signatures)



SECTION 4 — SUPER-PEER ELECTION & TOPOLOGY

5 questions

16

Describe the Bully election algorithm as implemented in MobiCloud. How does it avoid split-brain?

Hard

Bully Algorithm

Super-Peer

Distributed Consensus

◆ MODEL ANSWER

The election is **local and deterministic** — no extra round-trip is required:

```
// MobiCloudBullyElection.java (PeerSim simulation model)
// Each cycle: node scans neighbors' scores
if (higherSuperPeerExists) {
    this.isSuperPeer = false;           // Rule 1/2: stronger SP exists → abdicate
} else if (this.score >= maxNeighborScore) {
    this.isSuperPeer = true;           // Rule 3: I'm the best visible → become SP
}
```

- **Step 1:** Collect `ReliabilityScore` for all peers concurrently (via `async/awaitAll` in Android code)
- **Step 2:** Keep only peers with the maximum score
- **Step 3:** Tiebreak → **lexicographic max of nodeId** (deterministic — no extra message needed)
- **Split-brain prevention:** The algorithm is deterministic on the same input — everyone locally computes the same winner. No voting round, no coordinator. The relay server also enforces uniqueness: a second Super-Peer for the same `clusterId` is rejected if one is already connected (`REGISTER_PEER` logic in `server.js`)
- Super-Peer mandate: 30 minutes → automatic abdication → rotation prevents power concentration

17

How is the ReliabilityScore calculated and what factors does it measure?

Easy

Reliability Score

Node Ranking

✦ MODEL ANSWER

The ReliabilityScore is a composite [0.0, 1.0] metric recalculated every 30 seconds:

- **Uptime:** how long the node has been continuously available (stable nodes are preferred)
- **Network responsiveness:** round-trip time to other peers
- **Block transfer success rate:** successful uploads and downloads as a fraction of attempts
- **Network stability:** WiFi (stable) vs 4G (intermittent) — WiFi nodes get a higher coefficient
- Score is clamped to [0, 1] in the relay (`Math.min(1, Math.max(0, reliabilityScore))`)
- It is broadcast in the relay signaling registry (`reliabilityScore` field in `REGISTER_PEER`) so all cluster members can make independent, consistent election decisions

18

What happens when the Super-Peer crashes without sending a departure notice?

Medium

Failure Detection

Re-election

Heartbeat

✦ MODEL ANSWER

MobiCloud uses two complementary failure detection mechanisms:

- **WebSocket-level heartbeat:** The relay server sends a ping every 60 seconds (`HEARTBEAT_INTERVAL_MS = 60_000`); after 2 missed pongs (120s), the connection is declared dead → session is cleaned up
- **Application-level heartbeat:** Peers detect Super-Peer liveness loss after a timeout (15-second dead-peer detection per project metrics)
- **Re-election trigger:** When the Super-Peer's session closes on the relay, the relay broadcasts implicitly via the next `GET_PEERS` response which no longer lists the failed SP as connected
- **New election:** Remaining peers run the Bully algorithm → new Super-Peer inherits the cluster relay session and re-registers with `REGISTER_PEER`
- PeerSim simulation (`bully-churn-results.csv`) shows: after the Super-Peer is killed, re-election completes in **2-3 gossip cycles** (~4-6 seconds)

19

How does a new peer discover and join a cluster? What is the BitTorrent-style tracker role of the relay?

Easy

Peer Discovery

mDNS

Tracker

◆ MODEL ANSWER

Discovery is multi-mode depending on network context:

New peer joins:

- ↓ Sends JOIN (0x0B) to relay → announces presence without Super-Peer status
- ↓ Sends GET_PEERS (0x04) to relay → receives list of registered Super-Peers
- ↓ Connects to nearest/best Super-Peer → joins its cluster
- ↓ (LAN only) mDNS/UDP broadcast → discovers peers on the same WiFi network directly

- Only **Super-Peers announce themselves** via REGISTER_PEER (0x03) — acts like a BitTorrent tracker where only seeds/supernodes register
- Regular peers are invisible to the tracker — they consume the peer list but don't pollute it
- Cluster size is load-balanced: currentMemberCount is sent in REGISTER_PEER; the relay enforces MAX_CLUSTER_SIZE = 2 per cluster in the signaling registry
- JOIN deliberately resets isSuperPair=false to prevent ghost Super-Peer entries after reconnection

20

Your relay server is centralized — isn't that a contradiction for a P2P system?

Easy

Architecture

NAT Traversal

Trade-offs

♦ MODEL ANSWER

This is the **minimum viable centralization** justified by the constraints of mobile networks:

- **Role:** The relay is purely a *transport relay + signaling directory* — it stores no user data, no plaintext, no business logic
- **Zero-knowledge:** Blocs pass through encrypted and opaque; the relay cannot read them
- **Replaceable:** Any WebSocket-capable relay can be swapped in; the relay address is a configuration parameter
- **2-instance HA:** Deployed as 2 independent Node.js instances on Render — no single point of failure at the infrastructure level
- **No data at rest:** All relay state (`sessions` `relayBuffer` `signalingRegistry`) is in-process RAM — a relay crash loses nothing permanent
- Alternative (STUN/TURN) was considered but WebSocket relay offered simpler implementation with equivalent semantic guarantees for the mobile context



SECTION 5 — SIMULATION, SCALABILITY & PERFORMANCE

5 questions

21

What is PeerSim and why did you use it to validate MobiCloud's protocols instead of a real device test?

Easy

PeerSim

Simulation

Scalability Testing

◆ MODEL ANSWER

PeerSim is a **Java-based P2P network simulator** designed for large-scale distributed protocol evaluation:

- It allows simulating 10 to 10,000+ nodes on a single machine in seconds — impossible with real Android devices
- PeerSim uses a cycle-driven model: each simulation cycle corresponds to real protocol rounds (e.g., 2 seconds of gossip)
- MobiCloud implemented: `MobiCloudGossipDht.java` (gossip convergence), `MobiCloudBullyElection.java` (Bully re-election), `BatteryProtocol.java` (energy model), `ChurnControl.java` (random node departures)
- Results quantified: convergence curves, churn resilience, battery impact of different fan-out values, re-election speed after Super-Peer crash
- The PeerSim gossip model simplifies by removing the Bloom Filter (full DHT exchange) — the Android production code uses the full Bloom Filter delta-sync

22

What is the circuit breaker pattern and why is it critical for MobiCloud's repair subsystem?

Hard

Circuit Breaker

Churn

Resilience Pattern

♦ MODEL ANSWER

The circuit breaker is a **fault-tolerance pattern** from distributed systems that prevents cascading failures:

- **Problem scenario:** 50% of peers go offline simultaneously (e.g., mass WiFi disconnection). The repair system tries to re-replicate all affected blocks at once → network storm → the few surviving peers get overwhelmed → they crash too → full data loss
- **Solution:** The circuit breaker monitors churn over a 5-minute sliding window (`churnEvents` in `server.js`)
- **OPEN state (repairs suspended):** triggered when churn > 30% — prevents the repair storm
- **CLOSED state (repairs active):** restored when churn drops below 10% — hysteresis prevents oscillation
- The relay server also maintains `getChurnRate()` which computes unique departed nodes over the window as a fraction of total nodes
- This is the distributed systems equivalent of a home circuit breaker: accept a partial outage rather than risk total collapse

23

What were the real-world performance results of MobiCloud? What was the bottleneck?

Easy

Benchmark

4G / WiFi

IRL Test

✦ MODEL ANSWER

Real-world test validated end-to-end file transfer over 4G ↔ WiFi:

→ **Throughput:** bounded by 4G upload speed (~10 Mbps), not by any logic bottleneck in MobiCloud

→ **Connectivity matrix tested:**

→ 4G ↔ 4G: TCP direct ✓

→ 4G ↔ WiFi: WebSocket relay ✓

→ WiFi ↔ WiFi (different NAT): WebSocket relay ✓

→ WiFi ↔ WiFi (same LAN): TCP direct + mDNS ✓

→ Dead peer detection: 15 seconds

→ Repair scan: 10 seconds

→ Gossip convergence: 2-second interval, $O(\log N)$ cycles

→ The relay is deployed on Render with 2 HA instances tested in production

24

How does the battery energy model in PeerSim justify the choice of gossip fanout=2?

Medium

Battery Model

Fan-out

Trade-off Analysis

◆ MODEL ANSWER

The `BatteryProtocol.java` models radio energy consumption per gossip exchange:

```
// BatteryProtocol.java
costPerSend    = 0.5% // TX cost per gossip emission
costPerReceive = 0.3% // RX cost per gossip reception
// Node goes DOWN when batteryLevel ≤ 0
```

- **fanout=2 results:** 80%+ of nodes remain active over 4 minutes simulated — acceptable battery life
- **fanout=5 results:** Battery drains 2.5× faster; nodes exhaust in ~40 cycles (80 seconds simulated) — incompatible with mobile use
- **fanout=3 results:** Intermediate — faster convergence than fanout=2 but significant battery penalty
- Conclusion: **fanout=2 is the Pareto-optimal choice** — it achieves $O(\log N)$ convergence (sufficient for MobiCloud's 2-second gossip interval) while maximizing node availability
- This quantitative result justifies a key architectural decision in the production code

25

How does the relay server track and compute the churn rate? What data structure is used?

Medium

Churn Rate

Sliding Window

Monitoring

✦ MODEL ANSWER

The relay server implements a sliding-window churn tracker in pure in-memory JavaScript:

```
// server.js - churn tracking
const CHURN_WINDOW_MS = 5 * 60 * 1000; // 5-minute window
const churnEvents = []; // array of { ts, nodeId }

function recordChurn(nodeId) {
  churnEvents.push({ ts: Date.now(), nodeId });
  // Evict events older than 5 minutes (sliding window)
  while (churnEvents[0].ts < Date.now() - CHURN_WINDOW_MS) churnEvents.shift();
}

function getChurnRate() {
  const recent = churnEvents.filter(e => Date.now() - e.ts <= CHURN_WINDOW_MS);
  const uniqueDeparted = new Set(recent.map(e => e.nodeId));
  const total = signalingRegistry.size + uniqueDeparted.size;
  return Math.round((uniqueDeparted.size / total) * 100); // percentage
}
```

- Data structure: a sorted **FIFO array** as an approximation of a sliding window ($O(1)$ insert, $O(k)$ evict where k =events in expired window)
- Uses a **Set** to count *unique* departed nodes — prevents double-counting nodes that reconnected and left multiple times
- Cap at 10,000 events to bound memory

SECTION 6 — PROTOCOL DESIGN & BINARY FRAMING

3 questions

26

Describe the binary frame format used in the MobiCloud relay protocol. Why binary instead of JSON?

Medium

Binary Protocol

WebSocket

Performance

♦ MODEL ANSWER

Each WebSocket message follows a fixed binary frame format:

```
// buildFrame() / parseFrame() in server.js
Frame layout:
[0]      : opcode (1 byte, uint8) — e.g. 0x06 = UPLOAD
[1..4]   : payload length (4 bytes, uint32 LE)
[5..5+len]: payload (variable, ≤ 20 MB)

// For UPLOAD specifically:
payload: [0..15] = destNodeId (16 bytes UTF-8 NUL-padded)
        [16..79] = blockId    (64 bytes hex SHA-256)
        [80..]  = ciphertext (variable)
```

- **Why binary:** Block data is binary ciphertext — base64 encoding for JSON would add 33% size overhead; binary framing is zero-overhead
- **Deterministic parsing:** Fixed-offset fields avoid JSON parsing ambiguity and injection attacks
- **14 distinct opcodes** defined (0x01–0x0E, 0xFF) covering auth, signaling, transfer, gossip, election
- Validation in `parseFrame()`: exact length match, max size cap (20MB + 128 bytes header), prevents buffer overflow

27

How does the ELECTION_BROADCAST opcode prevent an election in cluster A from disturbing peers in cluster B?

Medium

Election Isolation

clusterId

Multi-cluster

◆ MODEL ANSWER

The `handleElectionBroadcast()` function in `server.js` filters by `clusterId`:

```
// server.js – handleElectionBroadcast()
// Extract clusterId from the JSON payload
let senderClusterId = JSON.parse(payload).clusterId ?? '';

for (const [nodeId, session] of sessions.entries()) {
  if (nodeId === fromNodeId) continue;
  if (senderClusterId) {
    const peerEntry = signalingRegistry.get(nodeId);
    // Only forward to nodes in the SAME cluster
    if (peerEntry?.clusterId && peerEntry.clusterId !== senderClusterId) continue;
  }
  safeSend(session.ws, buildFrame(MSG.ELECTION_RECEIVED, payload));
}
```

- Without this filter, a Bully ELECTION message from cluster A would be received by all nodes on the relay, causing spurious re-elections in unrelated clusters
- Legacy nodes (empty clusterId) receive all broadcasts for backward compatibility
- Cluster IDs are UUID v4 strings validated by regex: `/^[0-9a-f]{8}-[0-9a-f]{4}-4[0-9a-f]{3}-[89ab][0-9a-f]{3}-[0-9a-f]{12}$/i`

28

How does cross-cluster block retrieval work? What is the REQUEST_BLOCK opcode used for?

Hard

Inter-cluster

REQUEST_BLOCK

Pull Model

◆ MODEL ANSWER

`REQUEST_BLOCK` (0x0C) implements a pull-based inter-cluster block retrieval:

```
// handleRequestBlock() in server.js – Story 9.4
// Requester sends: REQUEST_BLOCK(destNodeId=16, blockId=64) – exactly 80 bytes
// Server pivots to destination: REQUEST_BLOCK_FORWARDED(fromNodeId=16, blockId=64)
```

Requester peer	Relay Server	Target peer
---REQUEST_BLOCK(0x0C)-->		
[destNodeId, blockId]		
	--REQUEST_BLOCK_FORWARDED	
	(0x0D)----->	
		(responds via UPLOAD)
<-----FORWARD (0x07)--	-----UPLOAD (0x06)--	

- **No buffering:** unlike UPLOAD, REQUEST_BLOCK is rejected immediately if the destination is offline (requester times out and retries via a different replica)
- **Anti-loop:** `destNodeId == fromNodeId` is rejected (case-insensitive comparison)
- Uses `Buffer.alloc(80)` (zero-initialized) not `allocUnsafe` — prevents memory leakage in the forwarded payload



SECTION 7 — APP ARCHITECTURE, DATABASE & ANDROID SPECIFICS

2 questions

29

Why is Room (SQLite) used for local storage instead of a full distributed database? What does each of the 7 DAOs manage?

Easy

Room / SQLite

DAO Pattern

Android Architecture

◆ MODEL ANSWER

Room is used as the **local persistence layer** — it represents only the data this device owns or indexes locally. The distributed database *is* the DHT itself.

- Room manages structured local state: DHT entries, hosted blocks, peer metadata, tombstones, file catalog, fragment location cache, and settings
- 7 DAOs across 8 entity types: `DhtEntryDao`, `HostedBlockDao`, `PeerDao`, `TombstoneDao`, `FileMetadataDao`, `FragmentLocationDao`, `LocalRepairBufferDao`
- Room provides compile-time SQL validation, LiveData/Flow reactive queries, and coroutine support — essential for the MVVM architecture
- DataStore (not Room) handles key-value preferences (nodeId, settings)
- The 8 Room entities + 7 DAOs provide exactly the persistence needed for the local DHT replica, peer registry, and repair queue — nothing more (no user data is stored here)

30

How does the Clean Architecture structure in MobiCloud enforce separation of concerns, especially regarding the DHT use cases?

Medium

Clean Architecture

MVVM

Dependency Injection

✦ MODEL ANSWER

MobiCloud uses Clean Architecture with 4 distinct layers:

presentation/	→ Jetpack Compose UI + ViewModels (state holders, no business logic)
domain/	→ UseCases + Models + Repository interfaces Pure Kotlin – ZERO Android dependencies (testable on JVM)
data/	→ Repository implementations, DAOs, JNI bridges → Adapters between domain and Android infrastructure
core/	→ ErasureCodec (JNI), CryptoPrimitives, HkdfSha256
di/	→ Dagger/Hilt modules wiring 16+ injection bindings

- The **domain layer** (pure Kotlin) contains all business rules: `InsertDhtEntryUseCase`, `LookupBlockLocationUseCase`, `RunBullyElectionUseCase`, `GenerateHashcashProofUseCase` etc.
- Domain use cases depend only on interfaces → swappable data sources (Room, in-memory, mock)
- This is why **65+ test classes** can exercise full business logic without an Android device
- Hilt with 16+ modules provides compile-time dependency injection across all layers
- Kotlin Coroutines + StateFlow + Flow handle reactive data propagation from data layer to UI without coupling

MobiCloud — Big Data Interview Questions & Answers · Generated from source code analysis · June 2026

Covers: `relay-server/server.js` · `simulation/peersim-mobicloud/src/*` · Domain use cases · Core algorithms